A4 Experiments

A4.1 Artificial Tasks

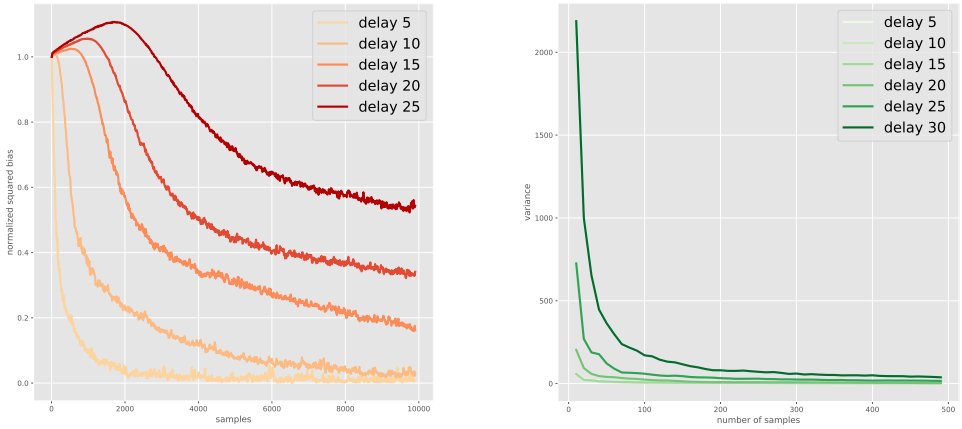
This section provides more details for the artificial tasks (I), (II) and (III) in the main paper. Additionally, we include artificial task (IV) characterized by deterministic reward and state transitions, and artificial task (V) which is solved using policy gradient methods.

A4.1.1 Task (I): Grid World

This environment is characterized by probabilistic delayed rewards. It illustrates a situation, where a time bomb explodes at episode end. The agent has to defuse the bomb and then run away as far as possible since defusing fails with a certain probability. Alternatively, the agent can immediately run away, which, however, leads to less reward on average since the bomb always explodes. The Grid World is a quadratic 31×31 grid with *bomb* at coordinate $[30, 15]$ and *start* at $[30 - d, 15]$, where d is the delay of the task. The agent can move in four different directions (*up*, *right*, *left*, and *down*). Only moves are allowed that keep the agent on the grid. The episode finishes after $1.5d$ steps. At the end of the episode, with a given probability of 0.5, the agent receives a reward of 1000 if it has visited *bomb*. At each time step the agent receives an immediate reward of $c \cdot t \cdot h$, where the factor c depends on the chosen action, t is the current time step, and h is the Hamming distance to *bomb*. Each move of the agent, which reduces the Hamming distance to *bomb*, is penalized by the immediate reward via $c = -0.09$. Each move of the agent, which increases the Hamming distance to *bomb*, is rewarded by the immediate reward via $c = 0.1$. The agent is forced to learn the Q-values precisely, since the immediate reward of directly running away hints at a sub-optimal policy.
For non-deterministic reward, the agent receives the delayed reward for having visited *bomb* with probability $p(r_{T+1} = 100 \mid s_T, a_T)$. For non-deterministic transitions, the probability of transiting to next state s' is $p(s' \mid s, a)$. For the deterministic environment these probabilities were either 1 or zero.

Policy evaluation: learning the action-value estimator for a fixed policy. First, the theoretical statements on bias and variance of estimating the action-values by TD in Theorem A8 and by MC in Theorem A10 are experimentally verified for a fixed policy. Secondly, we consider the bias and variance of TD and MC estimators of the transformed MDP with optimal reward redistribution according to Theorem A5.
The new MDP with an optimal reward redistribution has advantages over the original MDP both for TD and MC. For TD, the new MDP corrects the bias exponentially faster and for MC it has fewer number of action-values with high variance. Consequently, estimators for the new MDP learn faster than the same estimators in the original MDP.
Since the bias-variance analysis is defined for a particular number of samples drawn from a fixed distribution, we need to fix the policy for sampling. We use an ϵ-greedy version of the optimal policy, where ϵ is chosen such that on average in 10% of the episodes the agent visits *bomb*. For the analysis, the delay ranges from 5 to 30 in steps of 5. The true Q-table for each delay is computed by backward induction and we use 10 different action-value estimators for computing bias and variance.
For the TD update rule we use the exponentially weighted arithmetic mean that is sample-updates, with initial value $q^0(s, a) = 0$. We only monitor the mean and the variance for action-value estimators at the first time step, since we are interested in the time required for correcting the bias. 10 different estimators are run for 10,000 episodes. Figure A3a shows the bias correction for different delays, normalized by the first error.
For the MC update rule we use the arithmetic mean for policy evaluation (later we will use constant-α MC for learning the optimal policy). For each delay, a test set of state-actions for each delay is generated by drawing 5,000 episodes with the ϵ-greedy optimal policy. For each action-value estimator the mean and the variance is monitored every 10 visits. If every action-value has 500 updates (visits), learning is stopped. Bias and variance are computed based on 10 different action-value estimators. As expected from Section A3.1, in Figure A3b the variance decreases by $1/n$, where n is the number of samples. Figure A3b shows that the number of state-actions with a variance larger than a threshold increases exponentially with the delay. This confirms the statements of Theorem A10.

Learning the optimal policy. For finding the optimal policy for the Grid World task, we apply Monte Carlo Tree Search (MCTS), Q-learning, and Monte Carlo (MC). We train until the greedy policy reaches 90% of the return of the optimal policy. The learning time is measured by the number of episodes. We use *sample updates* for Q-learning and MC [128]. For MCTS the greedy policy uses 0 for the exploration constant in UCB1 [68]. The greedy policy is evaluated in 100 episodes



(a) (b)

Figure A3: (a) Experimental evaluation of bias and variance of different Q-value estimators on the Grid World. (b) Normalized bias reduction for different delays. Right: Average variance reduction for the 10th highest values.

intervals. The MCTS selection step begins in the start state, which is the root of the game tree that is traversed using UCB1 [68] as the tree policy. If a tree-node gets visited the first time, it is expanded with an initial value obtained by 100 simulated trajectories that start at this node. These simulations use a uniform random policy whose average Return is calculated. The backpropagation step uses the MCTS(1) update rule [66]. The tree policies exploration constant is $\sqrt{2}$. Q-learning and MC use a learning rate of 0.3 and an ϵ-greedy policy with $\epsilon = 0.3$. For RUDDER the optimal reward redistribution using a return decomposition as stated in Section A2.6.1 is used. For each *delay* and each method, 300 runs with different seeds are performed to obtain statistically relevant results.

Estimation of the median learning time and quantiles. The performance of different methods is measured by the median learning time in terms of episodes. We stop training at 100 million episodes. Some runs, especially for long delays, have taken too long and have thus been stopped. To resolve this bias the quantiles of the learning time are estimated by fitting a distribution using right censored data [33] .The median is still robustly estimated if more than 50% of runs have finished, which is the case for all plotted datapoints. We find that for delays where all runs have finished the learning time follows a Log-normal distribution. Therefore, we fit a Log-normal distribution on the right censored data. We estimate the median from the existing data, and use maximum likelihood estimation to obtain the second distribution parameter σ^2. The start value of the σ^2 estimation is calculated by the measured variance of the existing data which is algebraically transformed to get the σ parameter.

A4.1.2 Task (II): The Choice

In this experiment we compare RUDDER, temporal difference (TD) and Monte Carlo (MC) in an environment with delayed deterministic reward and probabilistic state transitions to investigate how reward information is transferred back to early states. This environment is a variation of our introductory pocket watch example and reveals problems of TD and MC, while contribution analysis excels. In this environment, only the first action at the very beginning determines the reward at the end of the episode.

The environment is an MDP consisting of two actions $a \in \mathcal{A} = \{+, -\}$, an initial state s^0, two *charged* states s^+, s^-, two *neutral* states $s^\oplus$, $s^\ominus$, and a final state s^{f}. After the first action $a_0 \in \mathcal{A} = \{+, -\}$ in state s^0, the agent transits to state s^+ for action $a_0 = +$ and to s^- for action $a_0 = -$. Subsequent state transitions are probabilistic and independent on actions. With probability p_C the agent stays in the *charged* states s^+ or s^-, and with probability $(1 - p_C)$ it transits from s^+ or s^- to the *neutral* states $s^\oplus$ or $s^\ominus$, respectively. The probability to go from *neutral* states to *charged* states is p_C, and the probability to stay in *neutral* states is $(1 - p_C)$. Probabilities to transit from s^+

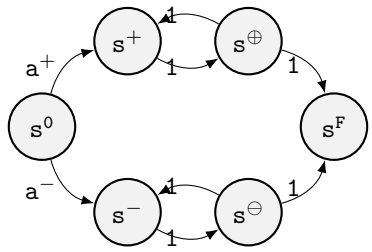


Figure A4: State transition diagram for The Choice task. The diagram is a simplification of the actual MDP.

or $s^{\oplus}$ to s^{-} or $s^{\ominus}$ or vice versa are zero. Thus, the first action determines whether that agent stays in "$+$"-states or "$-$"-states. The reward is determined by how many times the agent visits *charged* states plus a bonus reward depending on the agent's first action. The accumulative reward is given at sequence end and is deterministic. After T time steps, the agent is in the final state s^{f}, in which the reward R_{T+1} is provided. R_{T+1} is the sum of 3 deterministic terms:

1. R_0, the baseline reward associated to the first action;
2. R_C, the collected reward across states, which depends on the number of visits n to the *charged* states;
3. R_b, a bonus if the first action $a_0 = +$.

The expectations of the accumulative rewards for R_0 and R_C have the same absolute value but opposite signs, therefore they cancel in expectation over episodes. Thus, the expected return of an episode is the expected reward R_b: $p(a_0 = +)b$. The rewards are defined as follows:

$$c_0 \;=\; \begin{cases} 1 & \text{if } a_0 = + \\ -1 & \text{if } a_0 = - \,, \end{cases} \tag{A255}$$

$$R_b \;=\; \begin{cases} b & \text{if } a_0 = + \\ 0 & \text{if } a_0 = - \,, \end{cases} \tag{A256}$$

$$R_C \;=\; c_0 \, C \, n \,, \tag{A257}$$

$$R_0 \;=\; -\, c_0 \, C \, p_C \, T \,, \tag{A258}$$

$$R_{T+1} \;=\; R_C \;+\; R_0 \;+\; R_b \,, \tag{A259}$$

where C is the baseline reward for *charged* states, and p_C the probability of staying in or transiting to *charged* states. The expected visits of charged states is $\mathrm{E}[n] = p_C T$ and $\mathrm{E}[R_{T+1}] = \mathrm{E}[R_b] = p(a_0 = +)b$.

Methods compared: The following methods are compared:

1. Q-learning with eligibility traces according to Watkins [140],
2. Monte Carlo,
3. RUDDER with reward redistribution.

For RUDDER, we use an LSTM without lessons buffer and without safe exploration. Contribution analysis is realized by differences of return predictions. For MC, Q-values are the exponential moving average of the episode return. For RUDDER, the Q-values are estimated by an exponential moving average of the reward redistribution.

Performance evaluation and results. The task is considered as solved when the exponential moving average of the selection of the desired action at time $t = 0$ is equal to $1 - \epsilon$, where ϵ is the exploration rate. The performances of the compared methods are measured by the average learning time in the number of episodes required to solve the task. A Wilcoxon signed-rank test is performed between the learning time of RUDDER and those of the other methods. Statistical significance p-values are obtained by Wilcoxon signed-rank test. RUDDER with reward redistribution is significantly faster than all other methods with p-values $< 10^{-8}$. Table A1 reports the number of episodes required by different methods to solve the task. RUDDER with reward redistribution clearly outperforms all other methods.